



Rapport Technique et Fonctionnel

Master d'Excellence M1 IAOC

ROBOUIT

Simulation de Robot Intelligent dans un Labyrinthe

Membre	Rôle
Ali ACHENAN	Chef de projet / Dev.
Hajar BOUIH	Développeuse
Wassim TRACHLI	Développeur
Sara AZZAM	Développeuse
Yassine ZAIMI	Développeur

Encadrant : **M. Mohammed DAOUDI** - Professeur FSK

Année universitaire : 2025/2026

1. Présentation Générale du Projet

1.1. Description

Robouit est une application de simulation interactive développée en C++17 avec la bibliothèque SFML 2.5+. Elle permet de simuler le déplacement d'un robot autonome dans un labyrinthe en utilisant différents algorithmes de recherche de chemin et d'apprentissage automatique. L'application offre une interface utilisateur complète pour créer, éditer et sauvegarder des labyrinthes, ainsi que pour visualiser en temps réel les performances des algorithmes d'IA.

1.2. Objectifs

- Créer une plateforme de simulation interactive pour la navigation de robot en temps réel.
- Implémenter et comparer différents algorithmes de recherche de chemin (A*, Q-Learning, Deep Q-Learning).
- Fournir un éditeur de labyrinthes interactif avec sauvegarde/chargement au format JSON.
- Proposer une interface utilisateur intuitive avec visualisation des métriques d'apprentissage.
- Permettre la personnalisation complète : textures, thèmes, paramètres audio et de simulation.

1.3. Public Cible

Le projet s'adresse principalement aux étudiants en informatique et IA, aux enseignants souhaitant illustrer des algorithmes de navigation, aux chercheurs en robotique simulée et aux passionnés d'algorithmique.

1.4. Lien du Dépôt GitHub

Depot : <https://github.com/AliAch04/SFML-RoboUIT>

2. Architecture Technique

2.1. Stack Technologique

Tableau 2.1. Stack technologique du projet

Composant	Détail
Langage	C++17
Framework UI	SFML 2.5+

Sérialisation	SimpleJSON (format JSON)
Compilateur	MSVC - Visual Studio 2022
OS cible	Windows 10/11 (64-bit)
Résolution	1600x900 et supérieures

2.2. Contraintes Techniques

Tableau 2.2. Contraintes de performance et système

Composant	Détail
Processeur	2 GHz ou supérieur
RAM	4 GB minimum
Carte graphique	Compatible OpenGL 2.0
Fréquence d'images	60 FPS stable (55 FPS minimum)
Temps de réponse UI	< 100 ms
Consommation RAM	< 500 MB
Démarrage	< 3 secondes

3. Structure du Projet

Le projet adopte une architecture modulaire avec séparation claire entre la logique métier et la présentation. Chaque module correspond à un ensemble de fichiers .cpp/.h dédiés. Les répertoires principaux sont les suivants : **assets/** pour les ressources graphiques, audio et polices ; **includes/** pour les fichiers d'en-têtes globaux ; **mazes/** pour les labyrinthes JSON

3.1. Structure des Fichiers Source

Ci-dessous la représentation de la structure complète du projet :

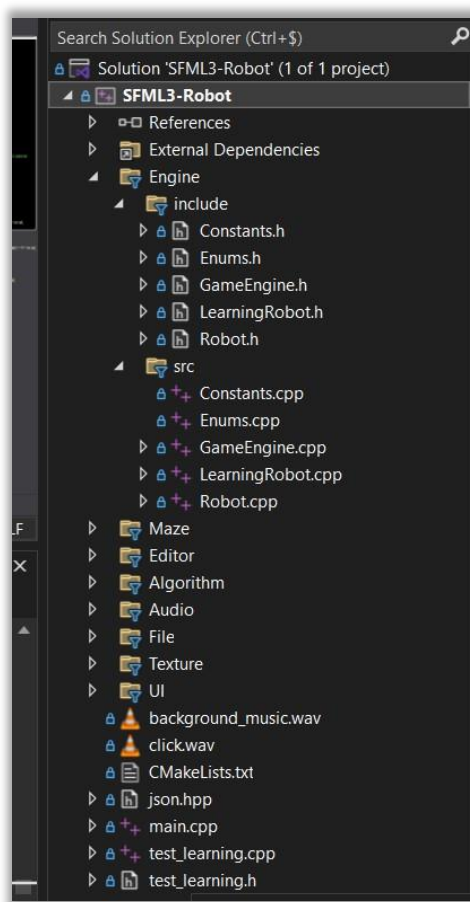


Figure 3.1. Structure du projet dans IDE Visual Studio Community 2022

3.2. Description des Modules

- Engine/ - Meteor de simulation (GameEngine, Robot, LearningRobot)
- Maze/ - Représentation du labyrinthe (Cell, Maze, Point)
- Editor/ - Editor de labyrinth (EditorToolbar, MazeEditor)
- Algorithm/ - Algorithms pathfinding et IA (AStar, QLearning, DeepQLearning, EvolutionaryAStar)

- UI/ - Composants interface (Button, Slider, ControlPanelWidget, PerformanceDashboard)
- Audio/ - Gestion audio (SoundManager)
- File/ - I/O fichiers, JSON, config, Logger
- Texture/ - Gestion des textures (TextureManager)

4. Installation et Configuration

4.1. Prérequis

Tableau 4.1. Logiciels requis pour l'installation

Composant	Détail
Windows	10/11 - 64-bit
Visual Studio	Community 2022 ou supérieur
SFML	2.5.1 - Visual C++ 17 (32-bit)
Git	Pour cloner le dépôt

4.2. Etapes d'installation

- Cloner le dépôt : `git clone https://github.com/AliAch04/SFML-RoboUIT.git`
- Télécharger SFML 2.5.1 depuis sfml-dev.org et extraire vers C:\SFML-2.5.1
- Ouvrir SFML3-Robot.sln dans Visual Studio 2022
- C/C++ > General > Répertoires d'inclusion : C:\SFML-2.5.1\include
- Editeur de liens > Répertoires de bibliothèques : C:\SFML-2.5.1\lib
- Copier les DLL depuis C:\SFML-2.5.1\bin vers le répertoire Debug/ ou Release/
- Se référer au README du dépôt GitHub pour plus de détails

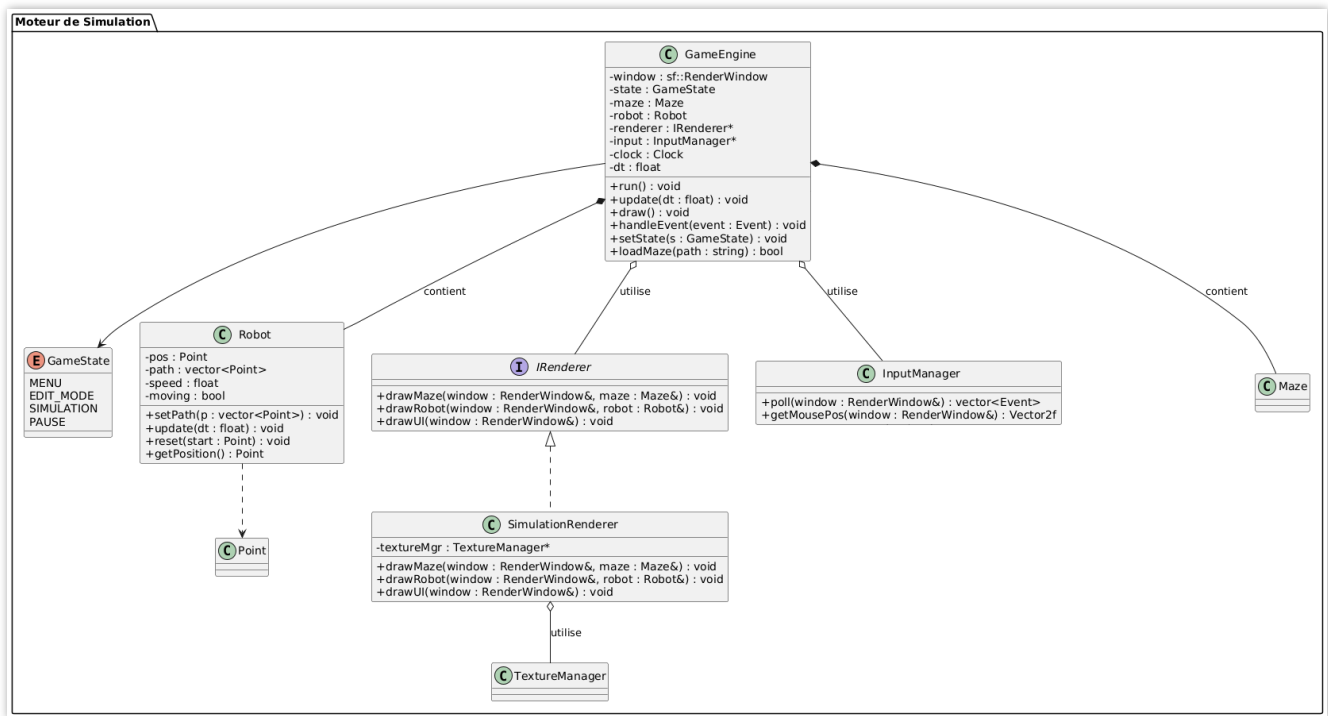


Figure 5.2. Diagramme de classes - Moteur de Simulation

Ce diagramme présente les classes principales du moteur : **GameEngine** (orchestre la boucle de jeu), **Robot** (position, état, animation) et **LearningRobot** (hérite de **Robot** avec des capacités d'apprentissage). Les relations d'héritage et de composition entre ces classes y sont clairement représentées.

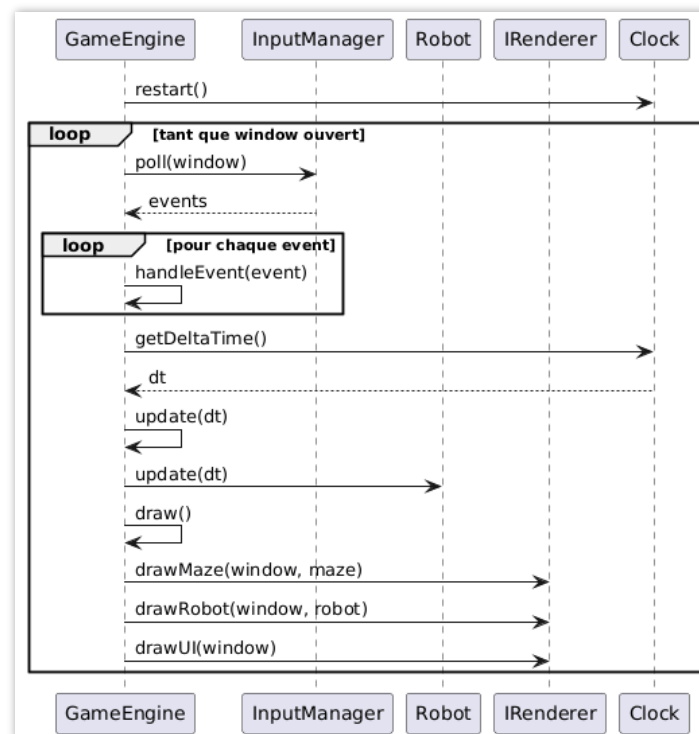


Figure 5.3. Diagramme de séquence - Moteur de Simulation

Ce diagramme décrit la séquence d'évènements lors du démarrage : l'utilisateur appuie sur Start, GameEngine initialise le Robot, le Robot calcule le chemin via l'algorithme sélectionné, puis les déplacements s'effectuent pas à pas jusqu'à l'état COMPLETED.

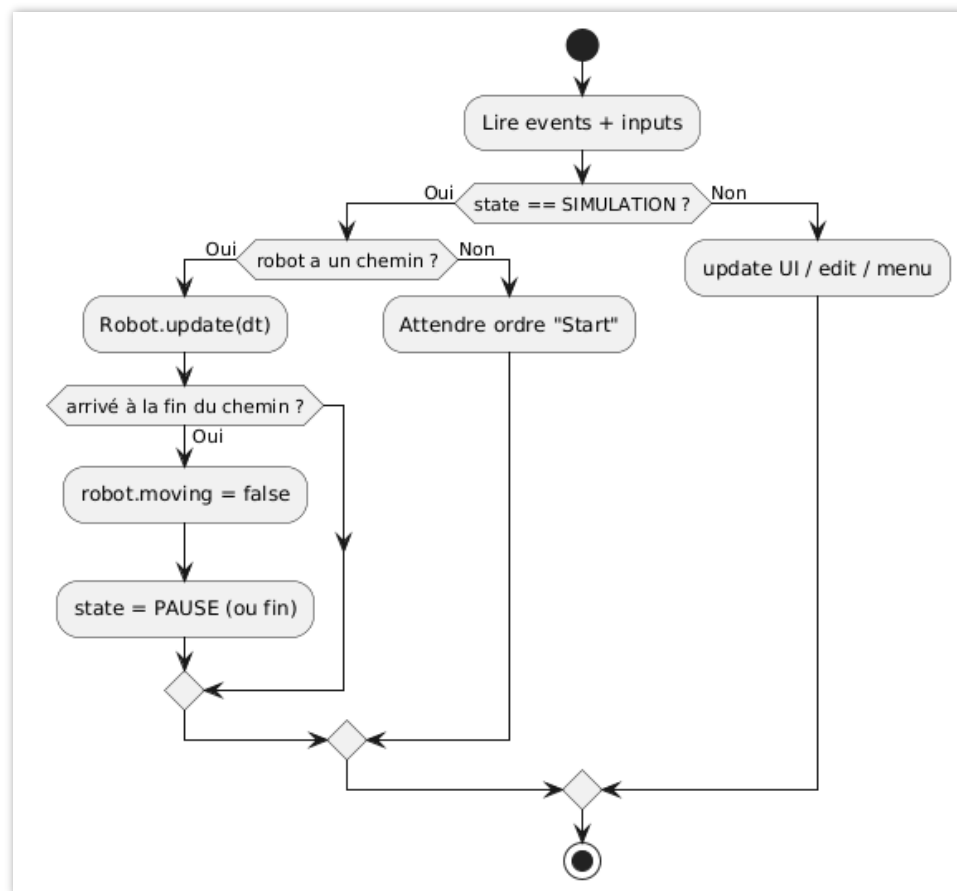


Figure 5.4. Diagramme d'activité - Moteur de Simulation

Ce diagramme modélise le flux du moteur : initialisation, boucle de déplacement cellule par cellule, gestion des transitions d'état (IDLE > MOVING > PAUSED > COMPLETED), et terminaison lorsque le robot atteint la case ARRIVEE.

5.2. Module 2 : Labyrinthe

Ce module gère la création, la génération aléatoire, le redimensionnement et la validation des labyrinthes. Il supporte les types de cellules MUR, VIDE, DEPART, ARRIVEE et SPECIAL.

Tableau 5.2. Besoins fonctionnels du module Labyrinthe

ID	Besoin	Description	Priorité
F05	Génération aléatoire	Création de labyrinthes solvables	Haute
F06	Redimensionnement	Changement dynamique des dimensions	Haute
F07	Types de cellules	MUR, VIDE, DEPART, ARRIVEE, SPECIAL	Haute
F08	Validation	Vérification de la solvabilité	Haute

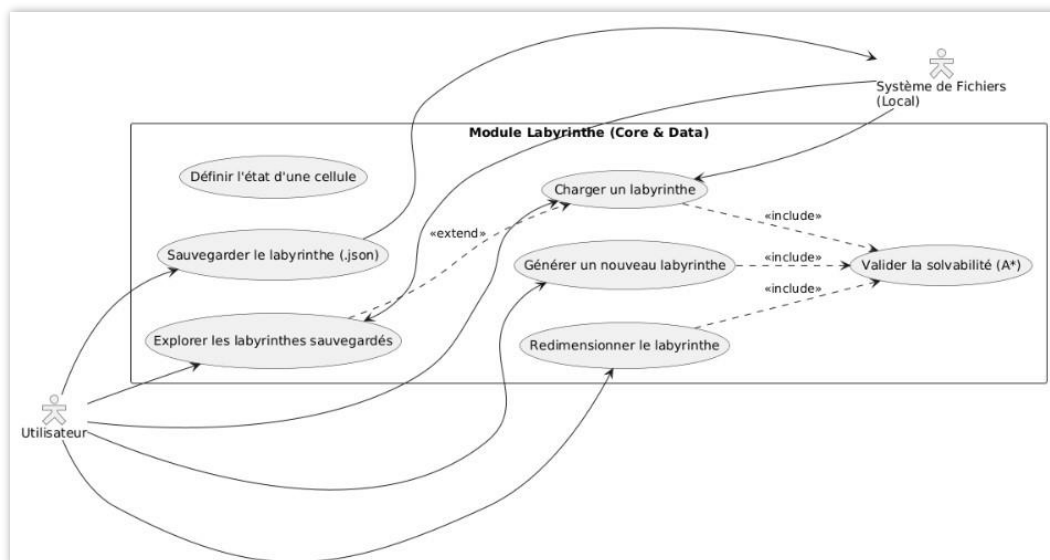


Figure 5.5. Diagramme de cas d'utilisation – Labyrinthe

Ce diagramme montre les interactions pour la gestion du labyrinthe : générer un labyrinthe aléatoire et solvable, redimensionner la grille dynamiquement, modifier les types de cellules (MUR, VIDE, DEPART, ARRIVEE, SPECIAL) et valider la solvabilité.

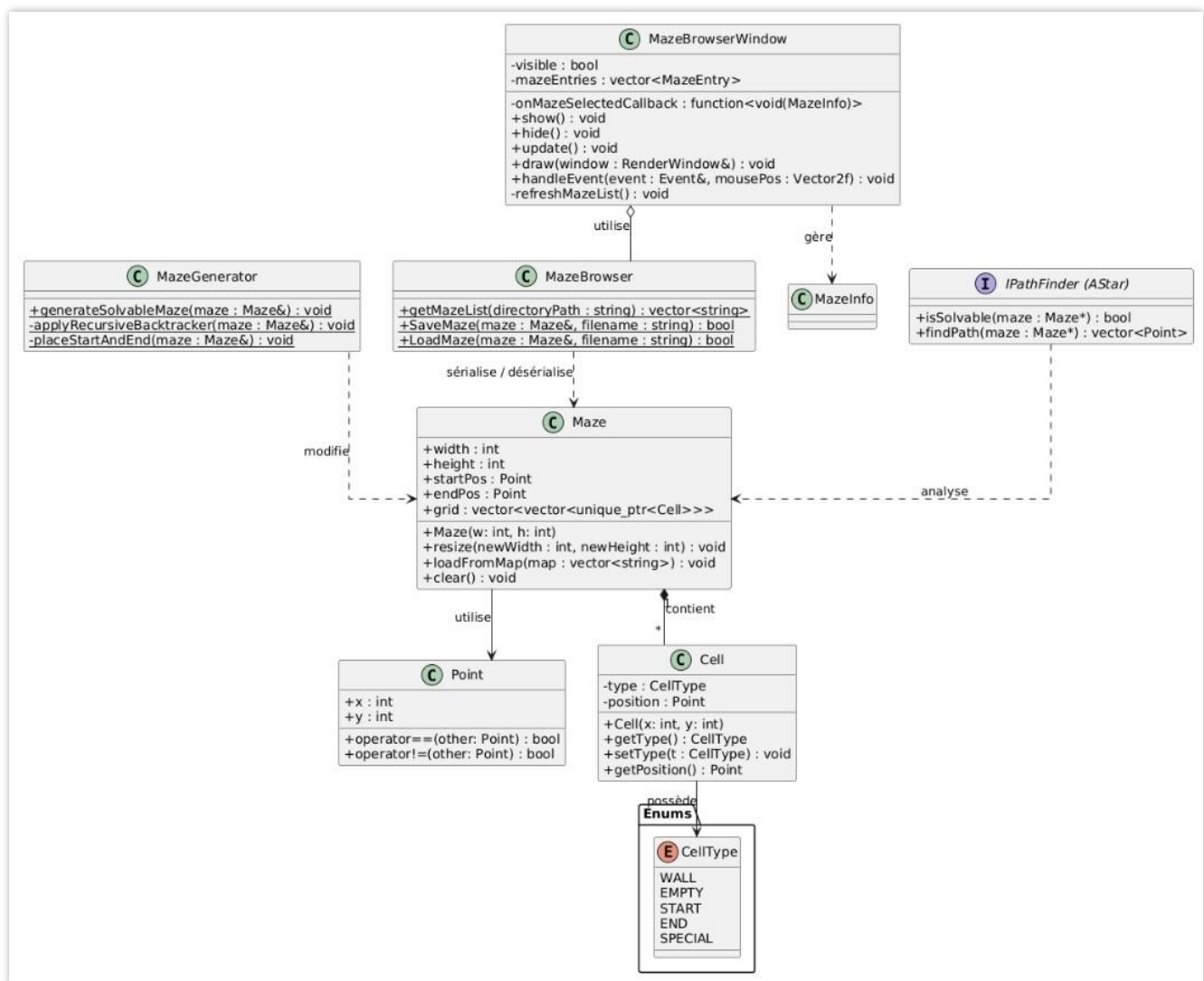


Figure 5.6. Diagramme de classes – Labyrinthe

Les classes Maze (grille de cellules), Cell (type et coordonnées) et Point (position x/y) sont représentées. Maze agrège les Cell et expose les opérations de génération (algorithme de perforation aléatoire) et de validation du chemin existant.

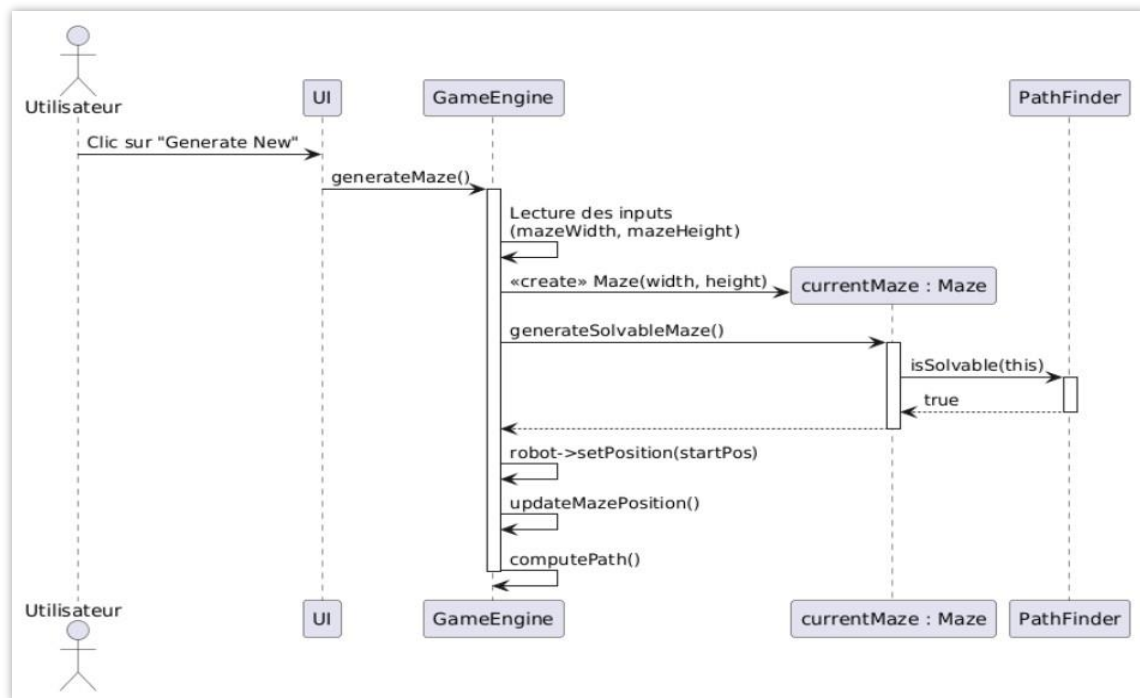


Figure 5.7. Diagramme de séquence – Labyrinthe

Ce diagramme illustre la séquence de génération : appel de `generateRandom()`, création des Cell, application de l'algorithme de génération, puis vérification de la solvabilité via un parcours en largeur (BFS) entre DEPART et ARRIVEE.

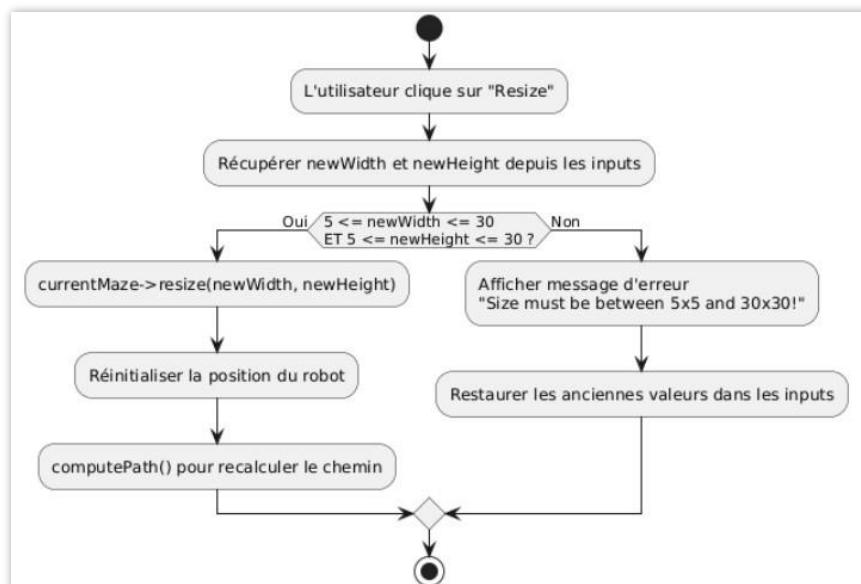


Figure 5.8. Diagramme d'activité – Labyrinthe

Le flux d'activité décrit les étapes : initialisation de la grille vide, génération aléatoire des murs, placement des cases de départ et d'arrivée, puis validation qu'un chemin existe entre les deux extrémités.

5.3. Module 3 : Editeur de Labyrinthe

L'éditeur permet à l'utilisateur de dessiner, effacer et placer des éléments dans le labyrinthe. Il intègre un système Undo/Redon, un aperçu ghost avant placement et un indicateur de l'outil actif.

Tableau 5.3. Besoins fonctionnels du module Editeur de Labyrinthe

ID	Besoin	Description	Priorité
F09	Outils de dessin	WALL, ERASE, START, END, SPECIAL	Haute
F10	Undo / Redon	Historique des modifications	Haute
F11	Ghost preview	Aperçu avant placement	Moyenne
F12	Indicateur de sélection	Highlight de l'outil actif	Moyenne

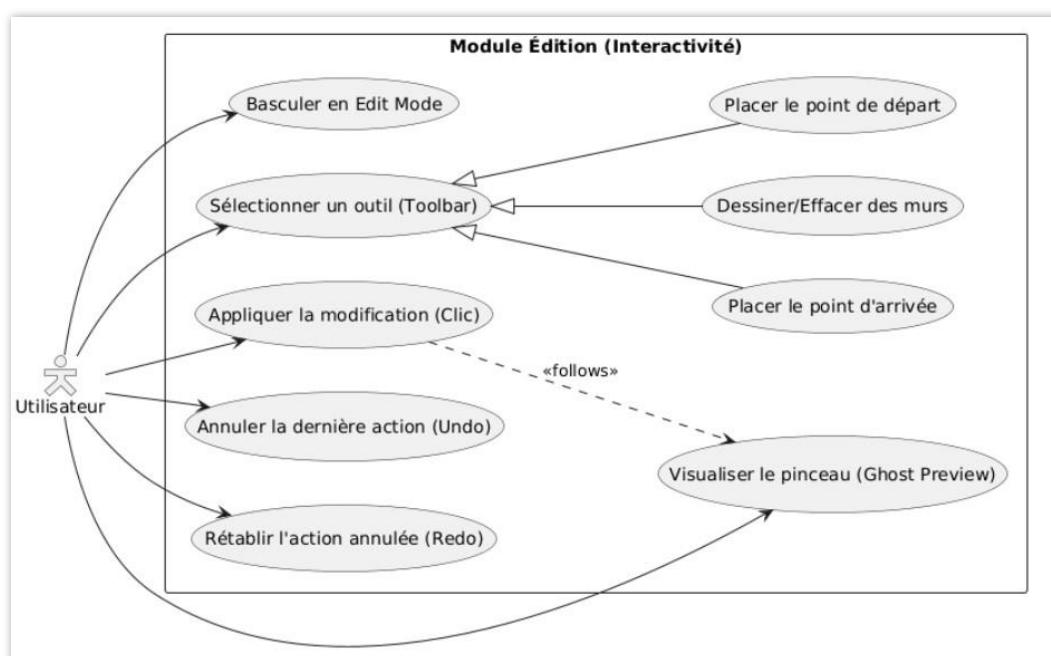


Figure 5.9. Diagramme de cas d'utilisation - Editeur de Labyrinthe

Ce diagramme présente les fonctionnalités de l'éditeur : sélectionner un outil (WALL, ERASE, START, END, SPECIAL), dessiner ou effacer des cellules, annuler/refaire des modifications (Undo/Redon), et visualiser un aperçu ghost avant le placement définitif.

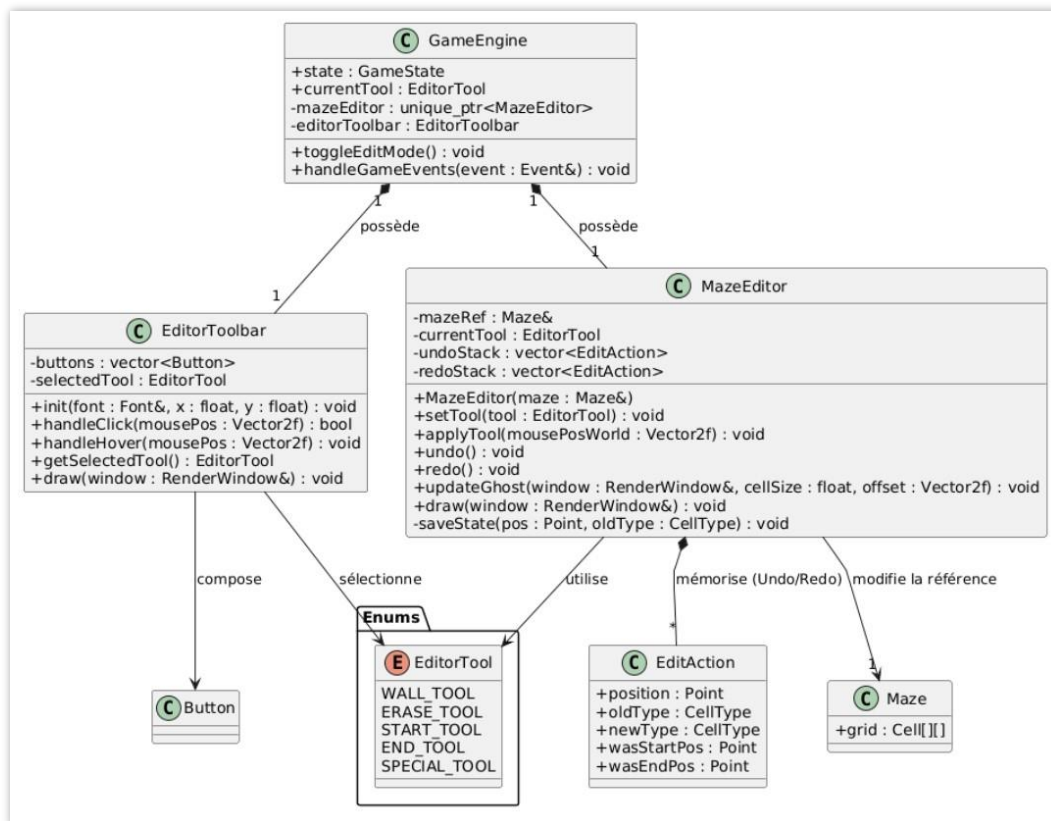


Figure 5.10. Diagramme de classes - Editeur de Labyrinthe

La classe EditorToolbar centralise la gestion des outils avec un historique de modifications base sur une pile (Undo/Redon). Elle interagit directement avec Maze pour appliquer les changements et maintient l'état de l'outil actif ainsi que l'aperçu de placement.

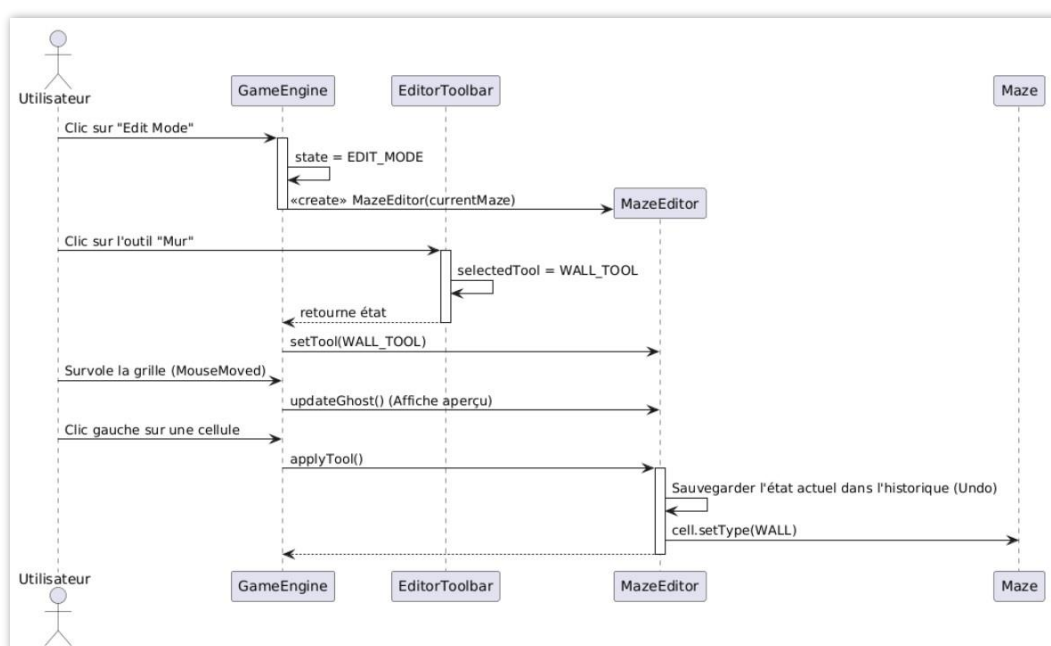


Figure 5.11. Diagramme de séquence - Editeur de Labyrinthe

La séquence illustre l'interaction lors d'un clic : sélection de l'outil actif, calcul de la cellule cible, sauvegarde de l'état précédent dans l'historique, application de la modification sur la grille, puis mise à jour de l'affichage.

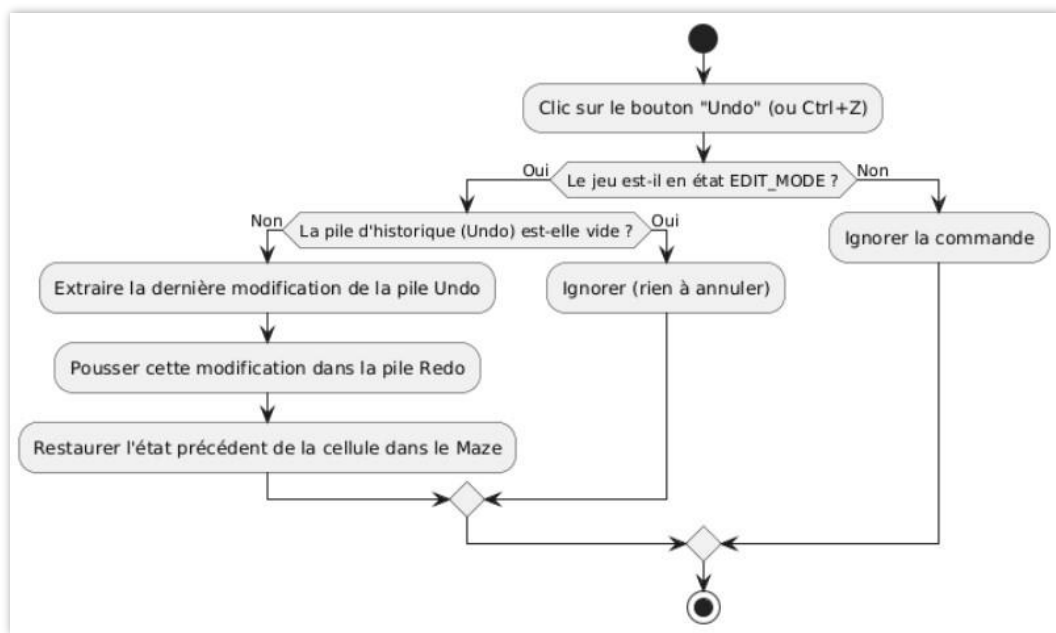


Figure 5.12. Diagramme d'activité - Editeur de Labyrinthe

Ce diagramme représente le flux de l'éditeur : attente d'une action utilisateur, sélection ou application d'un outil sur une cellule, enregistrement dans la pile historique, et rafraichissement de la vue avec l'aperçu ghost si applicable.

5.4. Module 4 : Algorithmes d'Intelligence Artificielle

Ce module est le cœur algorithmique du projet. Il implémente plusieurs stratégies de navigation : A* pour la recherche optimale, Q-Learning et Deep Q-Learning pour l'apprentissage par renforcement, ainsi qu'un algorithme évolutionnaire.

Tableau 5.4. Besoins fonctionnels du module Algorithmes d'IA

ID	Besoin	Description	Priorité
F13	A*	Recherche de chemin classique (optimal)	Haute
F14	Q-Learning	Apprentissage par renforcement tabulaire	Haute
F15	Deep Q-Learning	Réseaux de neurones artificiels	Moyenne
F16	Mode autonome	Le robot choisit ses propres actions	Haute
F17	Mode guide	Le robot suit le chemin calculé par A*	Haute

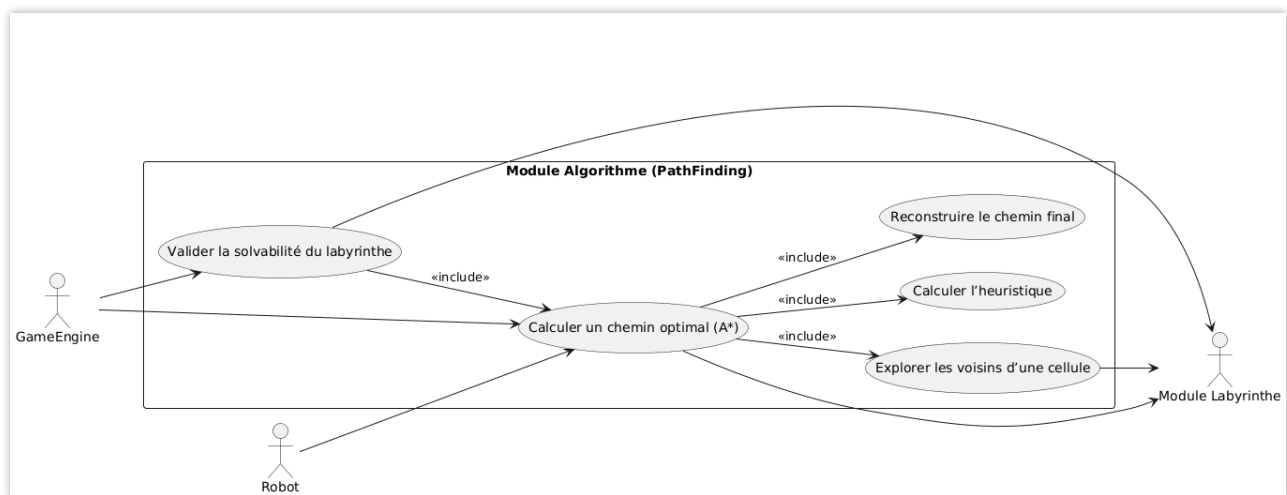


Figure 5.13. Diagramme de cas d'utilisation - Algorithmes d'IA

Ce diagramme couvre les cas d'utilisation algorithmiques : exécuter A* pour la recherche du chemin optimal, lancer Q-Learning pour l'apprentissage tabulaire, entraîner Deep Q-Learning via un réseau neuronal, ou activer l'algorithme évolutionnaire.

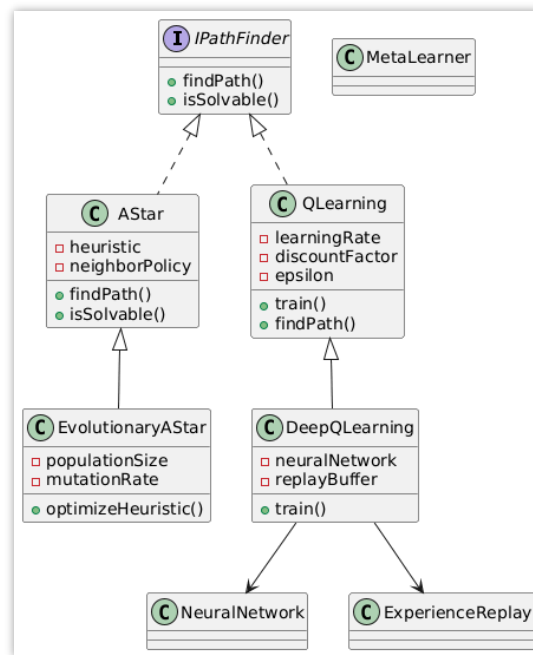


Figure 5.14. Diagramme de classes - Algorithmes d'IA

La hiérarchie montre : AStar et EvolutionaryAStar pour la recherche de chemin, QLearning pour l'apprentissage tabulaire, DeepQLearning avec NeuralNetwork et ExperienceReplay pour l'apprentissage profond. La classe Heuristics est partagée par les algorithmes de recherche.

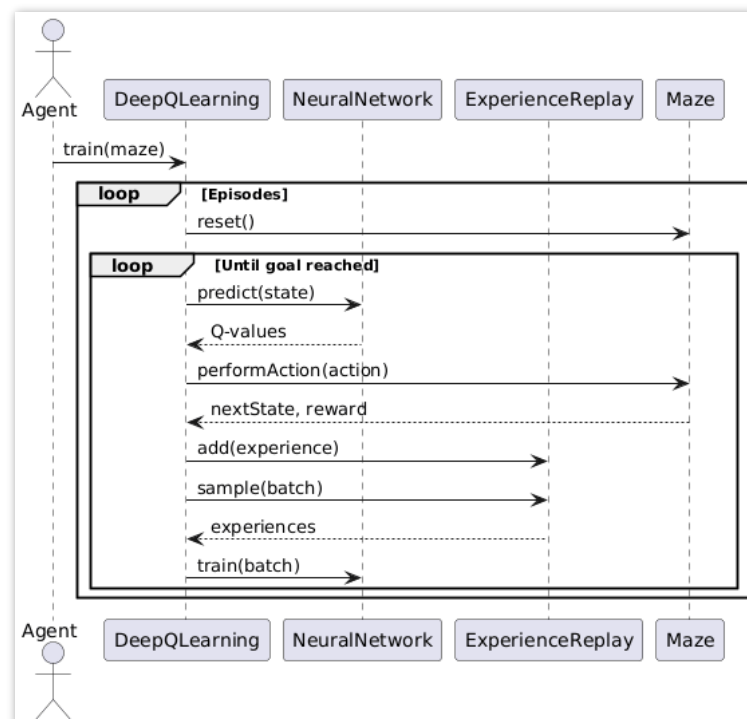


Figure 5.15. Diagramme de séquence - Algorithmes d'IA

Ce diagramme illustre l'exécution de A* : initialisation de la liste ouverte, évaluation de $f(n) = g(n) + h(n)$ pour chaque voisin, expansion du nœud de plus faible cout, reconstruction et restitution du chemin optimal jusqu'à l'arrivée.

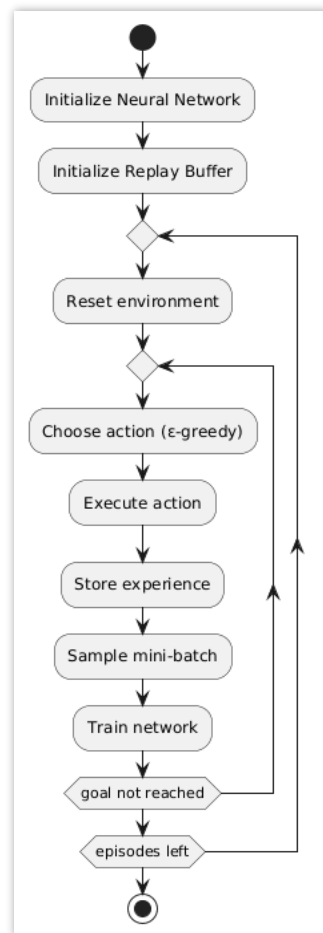


Figure 5.16. Diagramme d'activité - Algorithmes d'IA

Le diagramme modélise les boucles d'entrainement : pour Q-Learning, la boucle exploration/exploitation et mise à jour de la Q-table ; pour Deep Q-Learning, l'expérience replat et la retro propagation du réseau neuronal à chaque épisode.

5.5. Module 5 : Interface Utilisateur

L'interface est organisée en états (Menu principal, Options, Jeu). Elle comprend un panneau de contrôle, un Dashboard de métriques, des messages temporaires de notification et des tooltips contextuels.

Tableau 5.5. Besoins fonctionnels du module Interface Utilisateur

ID	Besoin	Description	Priorité
F18	Menu principal	Navigation START / OPTIONS / EXIT	Haute
F19	Options	Configuration vitesse, taille cellule	Haute
F20	Panneau de jeu	Boutons de contrôle de la simulation	Haute
F21	Dashboard	Visualisation des métriques	Moyenne
F22	Messages temporaires	Notifications utilisateur	Moyenne
F23	Tooltips	Aide contextuelle	Basse

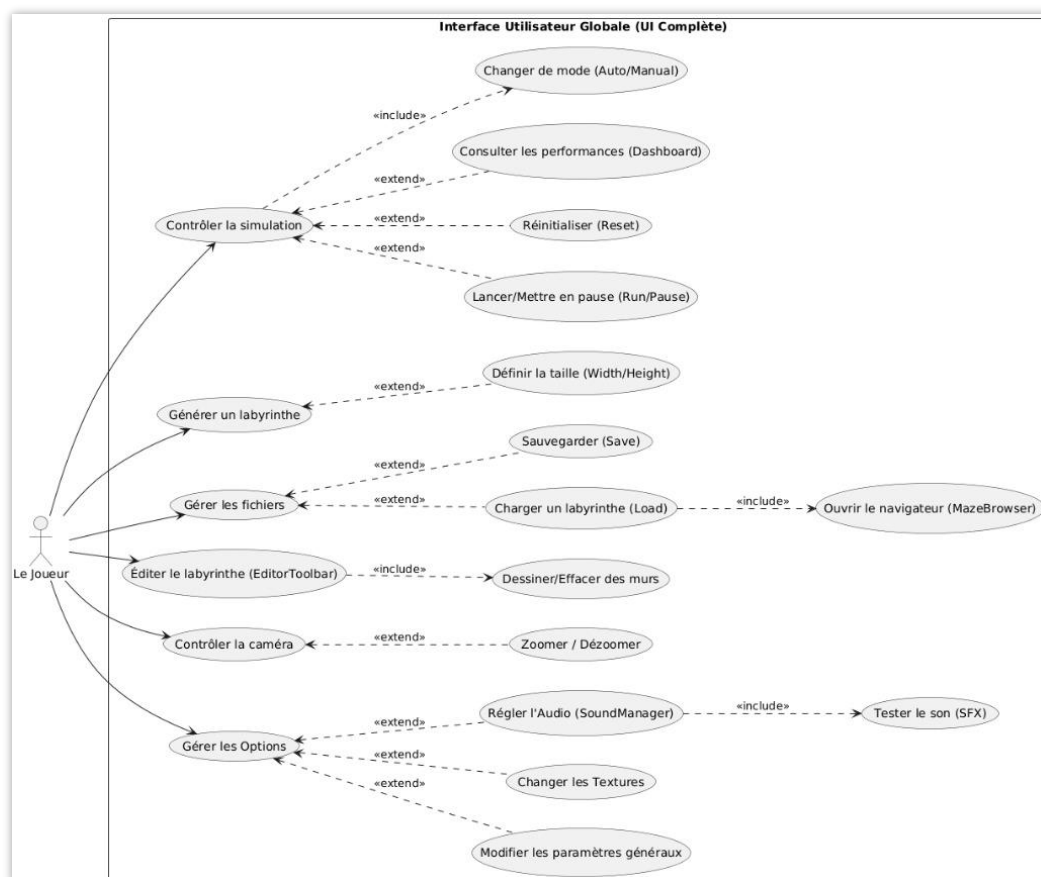


Figure 5.17. Diagramme de cas d'utilisation - Interface Utilisateur

Ce diagramme présente les interactions UI : naviguer dans les menus (START/OPTIONS/EXIT), configurer les paramètres (vitesse, taille des cellules), contrôler la simulation via le panneau de jeu, et consulter le Dashboard de métriques en temps réel.

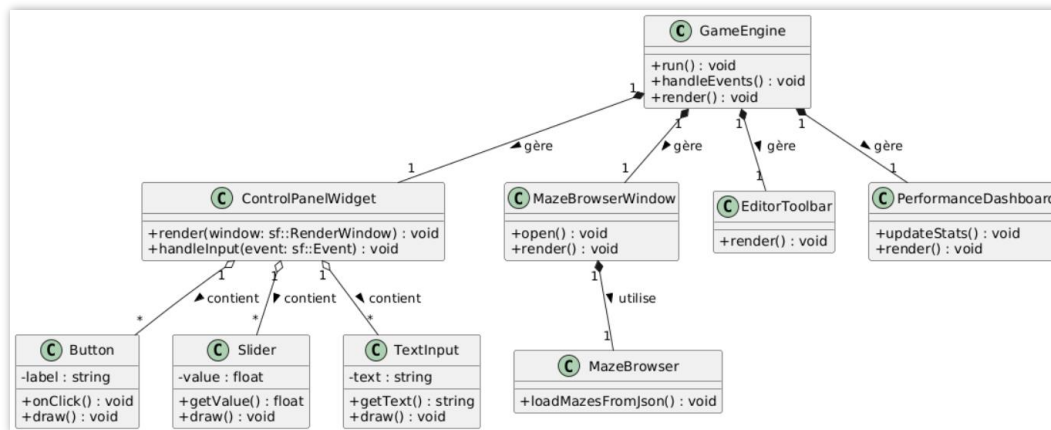


Figure 5.18. Diagramme de classes - Interface Utilisateur

Les composants UI sont : Button (interaction clic), Slider (valeurs continues), TitleBar (navigation globale), ControlPanelWidget (contrôles de simulation), PerformanceDashboard (métriques temps réel) et TrainingVisualizer (courbes d'apprentissage).

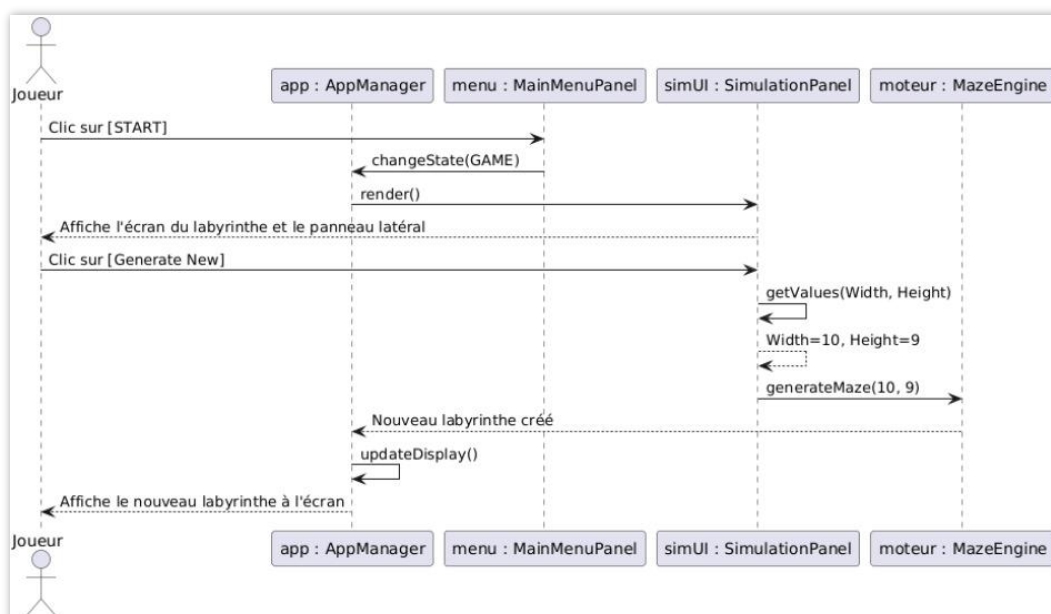


Figure 5.19. Diagramme de séquence - Interface Utilisateur

La séquence décrit le traitement d'un évènement SFML : capture du clic, identification du composant cible, appel du callback associé, mise à jour de l'état du jeu, puis rendu de la frame suivante a 60 FPS.

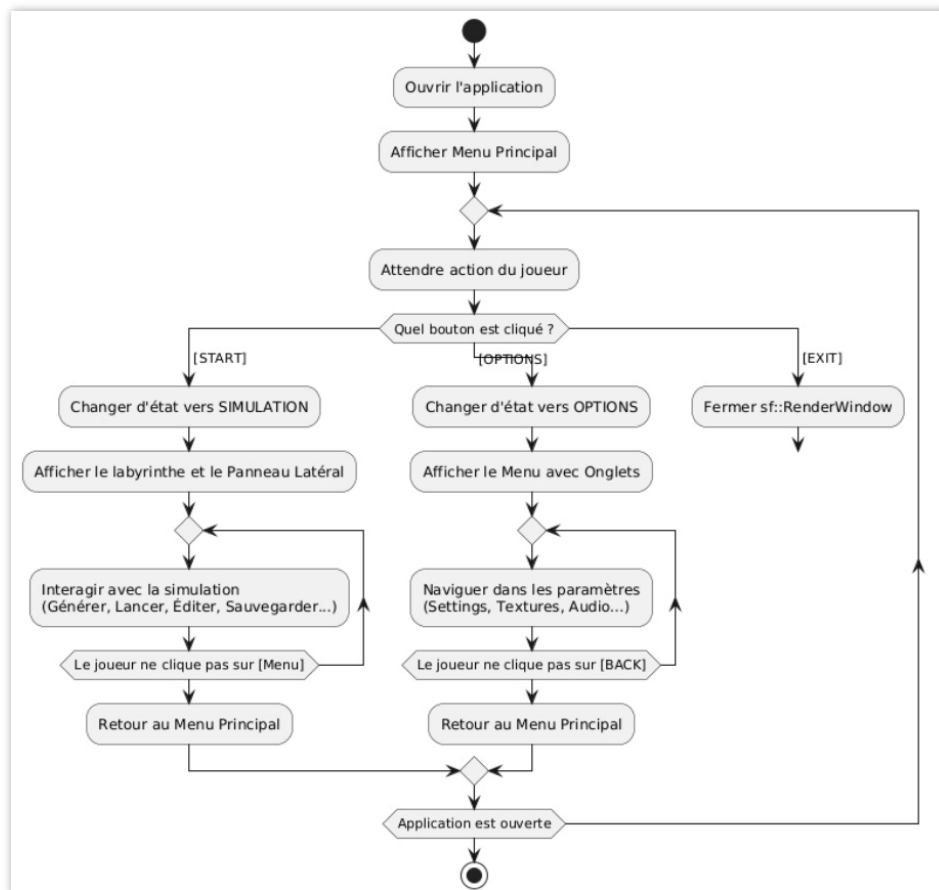


Figure 5.20. Diagramme d'activité - Interface Utilisateur

Ce diagramme modélise la machine d'état de l'interface : état MENU_PRINCIPAL, puis (START) état JEU ou (OPTIONS) état CONFIG, puis retour menu. Chaque état gère ses propres évènements et son propre rendu graphique.

5.6. Module 6 : Système Audio

Ce module gère la musique de fond en lecture continue, les effets sonores (clics, actions), les contrôles de volume avec sliders et les boutons de test audio.

Tableau 5.6. Besoins fonctionnels du module Système Audio

ID	Besoin	Description	Priorité
F24	Musique de fond	Lecture en continu	Moyenne
F25	Effets sonores	Clics, tests, actions	Moyenne
F26	Contrôle volume	Sliders Mute / Unmute	Moyenne
F27	Test audio	Boutons de test	Basse

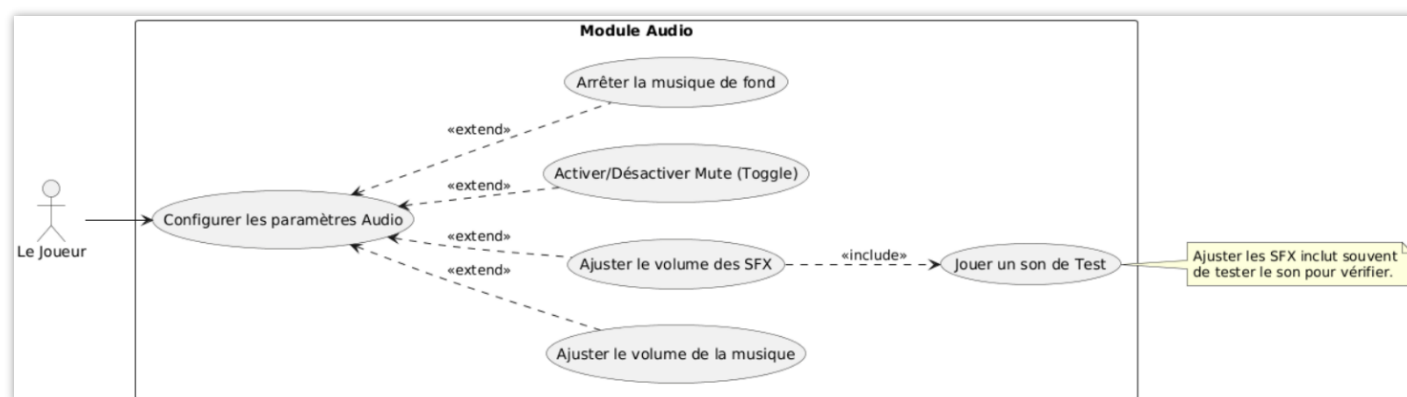


Figure 5.21. Diagramme de cas d'utilisation - Système Audio

Ce diagramme montre les interactions audio : activer ou couper la musique de fond, déclencher des effets sonores (clics, actions), ajuster le volume via slider, et tester les sons via les boutons dédiés de la page Options.

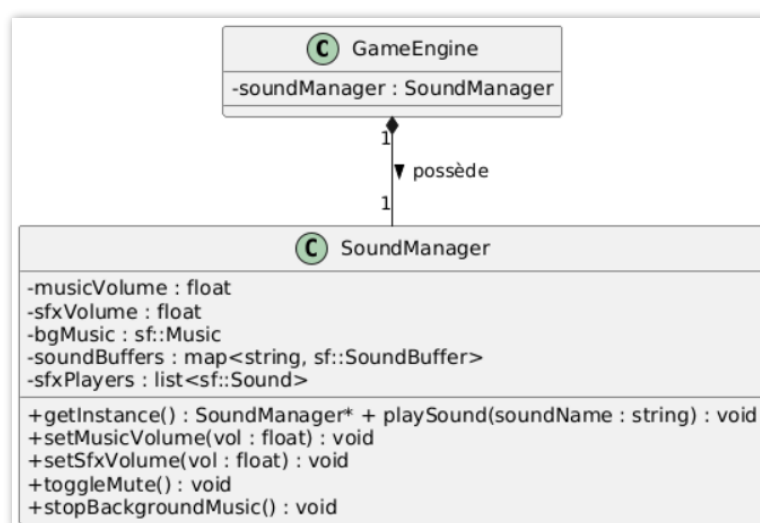


Figure 5.22. Diagramme de classes - Système Audio

La classe SoundManager centralise la gestion audio : sf::Music pour la musique de fond en streaming continu, sf::SoundBuffer et sf::Sound pour les effets ponctuels, avec contrôle de volume indépendant et gestion des états mute/unmute.

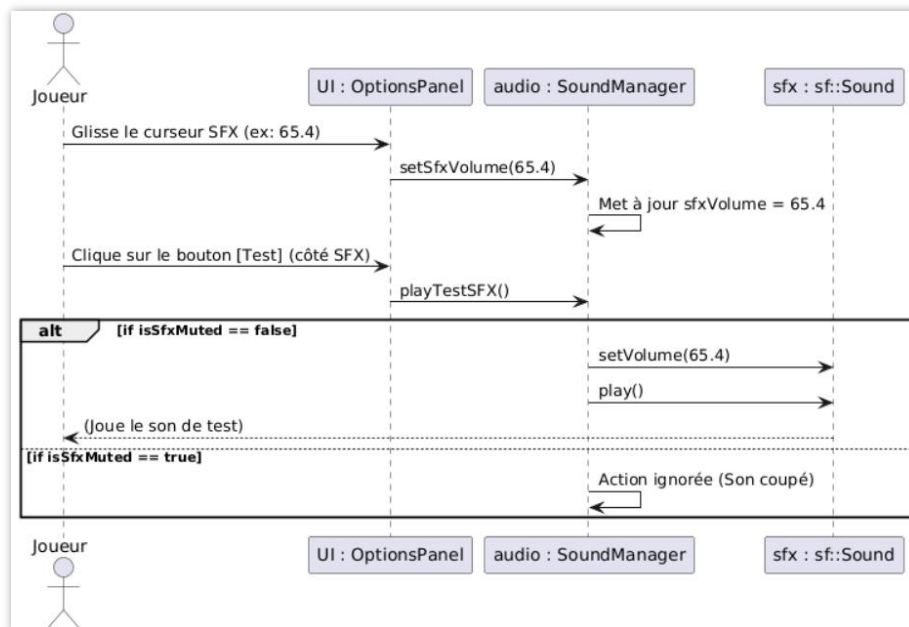


Figure 5.23. Diagramme de séquence - Système Audio

La séquence illustre le déclenchement d'un effet sonore : événement UI, appel playSound(id) sur SoundManager, chargement depuis le buffer en mémoire, puis lecture asynchrone du son sans interrompre la musique de fond.

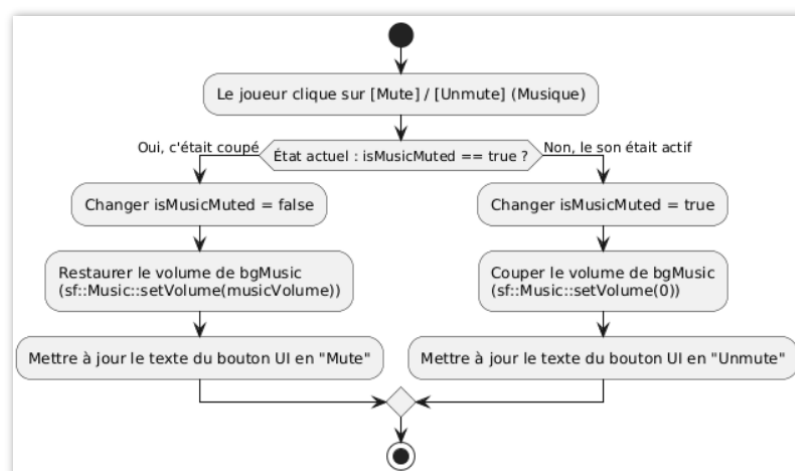


Figure 5.24. Diagramme d'activité - Système Audio

Le flux couvre : chargement initial des ressources audio au démarrage, boucle de lecture de la musique de fond, réponse aux événements volume/mute, et déclenchement asynchrone des effets sonores lors des actions utilisateur.

5.7. Module 7 : Gestion des Fichiers et Persistance

Ce module assure la sauvegarde et le chargement des labyrinthes au format JSON, la navigation dans les fichiers existants via un navigateur intègre, et la persistance des préférences utilisateur.

Tableau 5.7. Besoins fonctionnels du module Gestion des Fichiers

ID	Besoin	Description	Priorité
F28	Sauvegarde	Export du labyrinthe en JSON	Haute
F29	Chargement	Import depuis un fichier JSON	Haute
F30	Navigateur	Parcours des labyrinthes sauvegardés	Haute
F31	Configuration	Persistance des préférences utilisateur	Moyenne

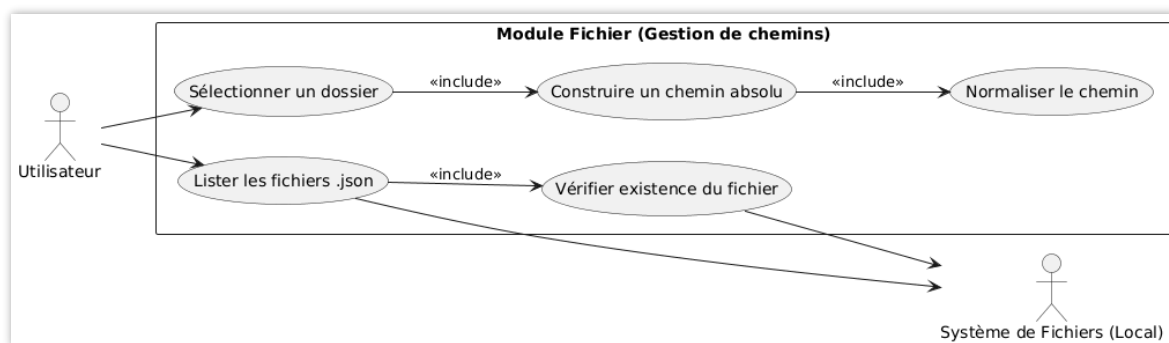


Figure 5.25. Diagramme de cas d'utilisation - Gestion des Fichiers

Ce diagramme présente les opérations de persistance : sauvegarder un labyrinthe au format JSON, charger un fichier existant, parcourir les labyrinthes sauvegardés via le navigateur intègre, et gérer la configuration persistante des préférences utilisateur.

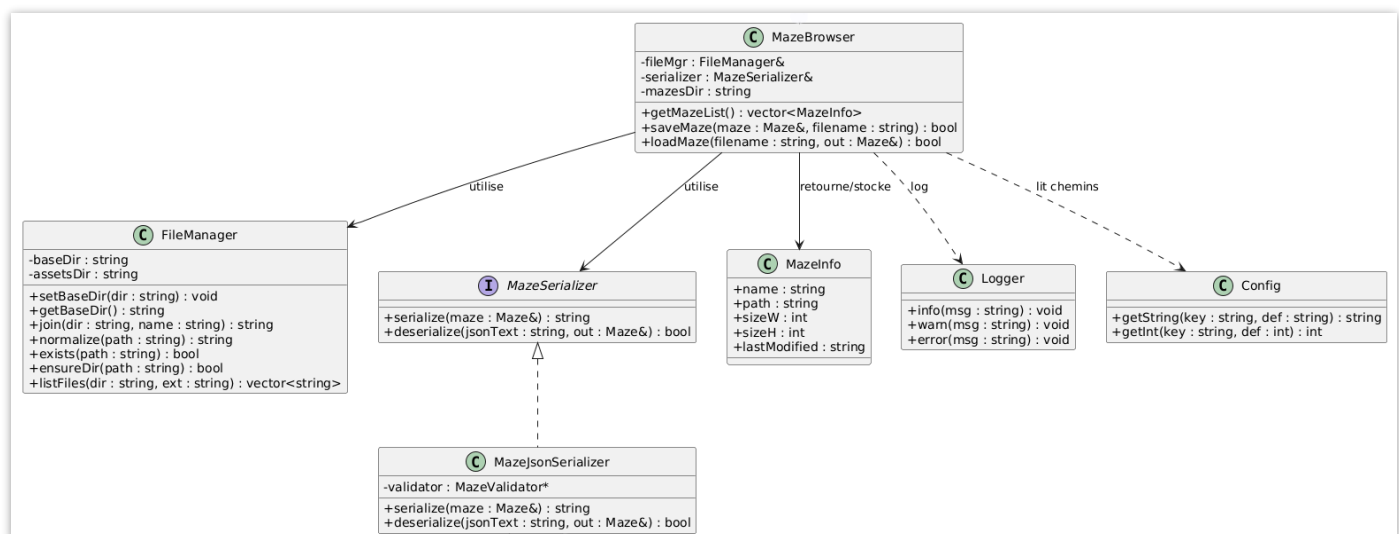


Figure 5.26. Diagramme de classes - Gestion des Fichiers

SimpleJSON gère la sérialisation/de sérialisation du labyrinthe. MazeBrowser liste et filtre les fichiers disponibles. Config persiste les préférences utilisateur, et Logger trace toutes les opérations pour faciliter le débogage et la maintenance.

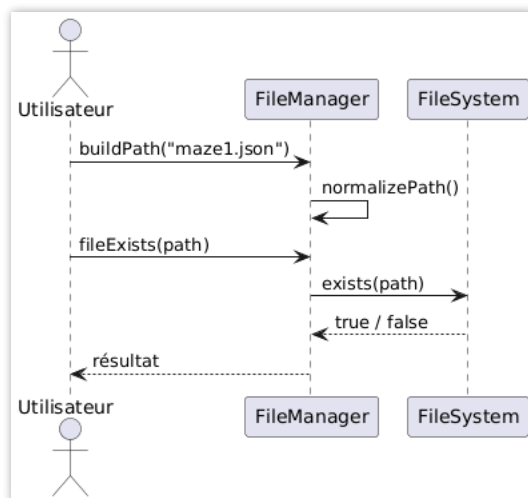


Figure 5.27. Diagramme de séquence - Gestion des Fichiers

La séquence de sauvegarde : l'utilisateur clique Save, saisie du nom de fichier, Maze sérialise ses données via SimpleJSON, écriture du fichier .json dans data/mazes/, puis confirmation affichée a l'utilisateur.

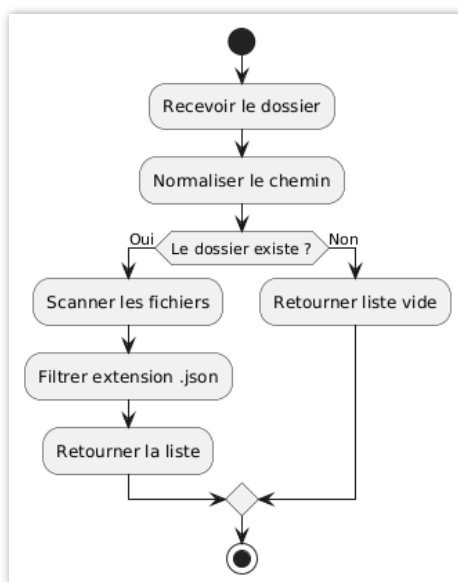


Figure 5.28. Diagramme d'activité - Gestion des Fichiers

Le flux d'activité décrit le chargement : ouverture du navigateur MazeBrowser, sélection d'un fichier, lecture et analyse JSON, de sérialisation, reconstruction du Maze en mémoire, puis affichage dans la vue principale.

5.8. Module 8 : Gestion des Textures

Ce module permet à l'utilisateur de personnaliser les textures de chaque élément du labyrinthe (murs, sol, robot, cases spéciales), de revenir aux textures par défaut et de les appliquer dynamiquement sans redémarrage.

Tableau 5.8. Besoins fonctionnels du module Gestion des Textures

ID	Besoin	Description	Priorité
F32	Personnalisation	Choix de textures pour chaque élément	Haute
F33	Reset	Retour aux textures par défaut	Moyenne
F34	Chargement dynamique	Application sans redémarrage	Haute

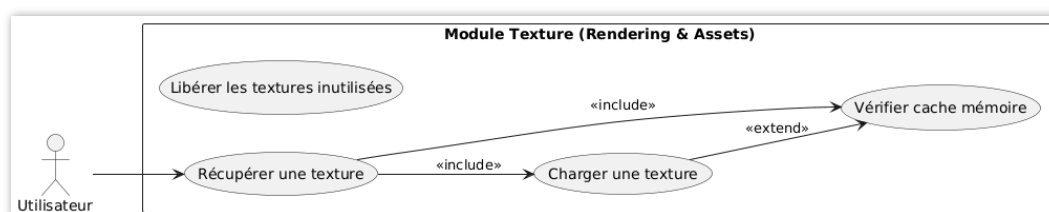


Figure 5.29. Diagramme de cas d'utilisation - Gestion des Textures

Ce diagramme couvre : choisir une texture personnalisée pour chaque élément (mur, sol, robot, cases spéciales), réinitialiser aux textures par défaut, et appliquer les changements dynamiquement sans redémarrage de l'application.

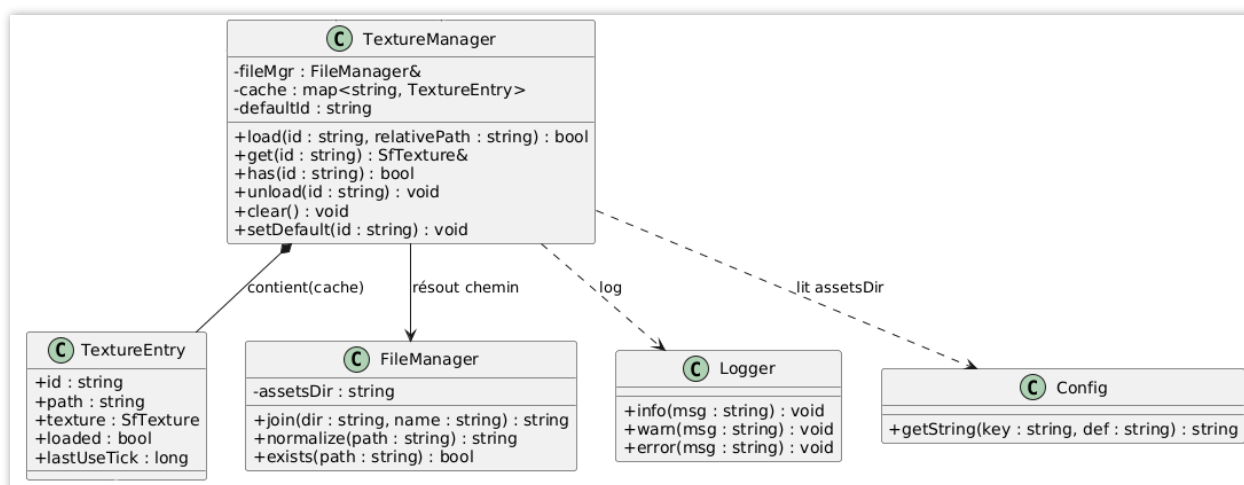


Figure 5.30. Diagramme de classes - Gestion des Textures

La classe **TextureManager** maintient une **map** d'identifiants vers des instances **sf::Texture**. Elle expose des méthodes de chargement depuis fichier, de récupération par identifiant et de réinitialisation. Le rendu SFML accède aux textures via cette interface centralisée.

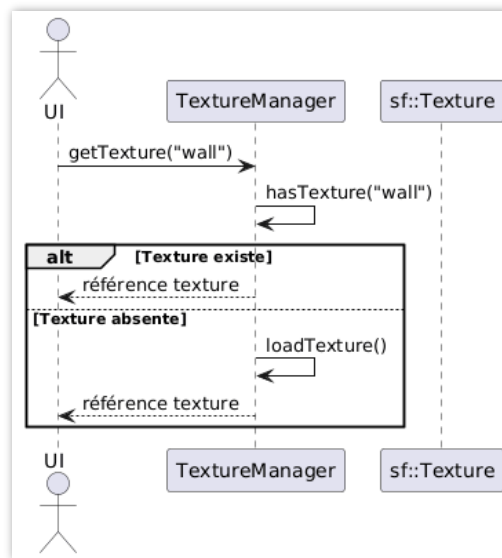


Figure 5.31. Diagramme de séquence - Gestion des Textures

Séquence de personnalisation : l'utilisateur sélectionne un élément et choisit une image, TextureManager::loadFromFile() charge la texture, la référence est mise à jour dans la mA, puis le prochain rendu utilise immédiatement la nouvelle texture.

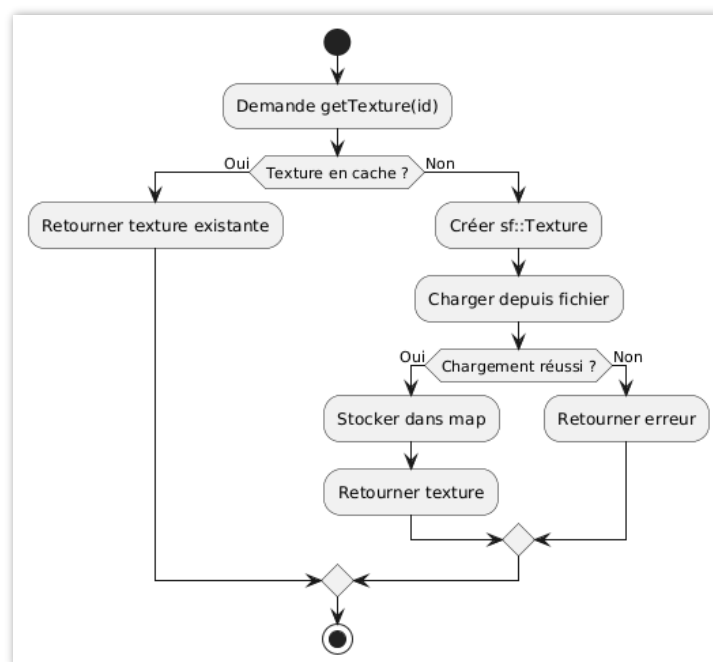


Figure 5.32. Diagramme d'activité - Gestion des Textures

Le flux d'activité décrit : initialisation avec les textures par défaut, sélection de l'élément à personnaliser, validation du fichier image, chargement dynamique en mémoire, et application immédiate dans la vue courante sans interruption.

6. Exigences Non Fonctionnelles

Tableau 6.1. Exigences non fonctionnelles du système

ID	Catégorie / Besoin	Critère d'acceptation
NF01	Performance - Fluidité	60 FPS constants (55 minimum)
NF02	Performance - Temps de réponse	< 100 ms pour les interactions UI
NF03	Performance - Mémoire	< 500 MB de RAM
NF04	Performance - Démarrage	< 3 secondes
NF05	Utilisabilité - Intuitivité	Interface compréhensible sans documentation
NF06	Utilisabilité - Feedback	Indications claires des actions effectuées
NF07	Fiabilité - Robustesse	Aucun crash sur actions utilisateur normales
NF08	Fiabilité - Gestion erreurs	Messages explicites en cas d'erreur
NF09	Maintenabilité - Modularité	Séparation claire des 8 modules
NF10	Extensibilité	Ajout de nouveaux algorithmes sans refonte

7. Organisation du Projet

7.1. Répartition des tâches

Tableau 7.1. Répartition des modules et tâches par membre

Membre	Rôle	Modules / Tâches assignées
Ali ACHENAN	Chef de projet	Architecture, Meteor (GameEngine, Robot), Algorithms IA (A*, DQL, Evolutionary*), coordination, TrainingVisualizer, Tests unitaires
Hajjar BOUIH	Développeuse	Interface Utilisateur (Button, Slider, TitleBar, ControlPanelWidget), Editeur de labyrinthe
Wassim TRACHLI	Développeur	Gestion Textures (TextureManager)
Sara AZZAM	Développeuse	Interface Utilisateur, System Audio
Yassine ZAIMI	Développeur	Gestion Fichiers/JSON

7.2. Organisation via GitHub

Le projet a été géré via GitHub (dépôt SFML-RoboUIT) en structurant le travail autour de 4 milestones et plus de 20 issues fermées. La progression est visible sur les graphes de contribution (Déc. 2025 - Fév. 2026), avec Ali ACHENAN en chef de projet (149 commits), Wassim TRACHLI (14 commits), Hajjar BOUIH (6 commits), Sara AZZAM (5 commits) et Yassine ZAIMI (1 commit).

- **Milestone Evolutionary A* Algorithm** : Implémentation de l'algorithme évolutionnaire et de l'optimisation long-terme (Jan. 2026).
- **Milestone Core Engine Improvements** : Améliorations du moteur principal et gestion audio (Jan. 2026).
- **Milestone Maze Editor Feature** : Implémentation complète de l'éditeur interactif et du navigateur de labyrinthes (Fév.. 2026).
- **Milestone Enhanced UI Architecture** : Refonte de la structure UI (menus, options, textures) avec corrections finales (Jan.-Fév.. 2026).



Figure 7.1. Graphe de contributions GitHub - SFML-RoboUIT (Déc. 2025 - Fév. 2026)

Conclusion Générale

Robouit est une plateforme complète de simulation de robot intelligent qui allie une interface utilisateur moderne et des algorithmes d'IA avancés. Grâce à ses 8 modules fonctionnels bien définis, elle offre une expérience utilisateur riche avec éditeur interactif, visualisation temps réel et feedback audio et visuel.

Le système met en œuvre des capacités d'apprentissage automatique avancées avec Q-Learning, Deep Q-Learning et l'algorithme évolutionnaire A*, le tout dans une architecture modulaire facilement extensible. Les performances optimales (60 FPS, animations fluides, démarrage en moins de 3 secondes) et la personnalisation poussée en font un outil parfaitement adapté au contexte universitaire.

Le projet répond pleinement aux objectifs pédagogiques et de recherche en fournissant un environnement visuel clair pour comprendre, tester et comparer différents algorithmes de navigation autonome. Il constitue un outil intermédiaire entre les solutions trop simples et les environnements professionnels complexes, assurant ainsi un apprentissage progressif et concret pour les étudiants du Master IAOC.

Bibliographie

- SFML Team. SFML 2.5 Documentation. <https://www.sfml-dev.org/documentation/2.5.1/>
- Sutton, R. S., Barto, A. G. Reinforcement Learning: An Introduction. MIT Press, 2018.
- Russell, S., Norvig, P. Artificial Intelligence: A Modern Approach. Pearson, 4e edition, 2020.
- ISO/IEC. ISO/IEC 14882:2017 - Programming Language C++. 2017.
- GitHub repository : <https://github.com/AliAch04/SFML-RoboUIT>